

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 8

1 - Enunciado

Implemente estimativa estocástica de π usando `rand()` para gerar os pontos. Cada thread deve usar uma variável privada para contar os acertos e acumular o total em uma variável global com `#pragma omp critical`. Depois, implemente uma segunda versão em que cada thread escreve seus acertos em uma posição distinta de um vetor compartilhado. A acumulação deve ser feita em um laço serial após a região paralela. Compare o tempo de execução das duas versões. Em seguida, substitua `rand()` por `rand_r()` em ambas e compare novamente. Explique o comportamento dos quatro programas com base na coerência de cache e nos efeitos do falso compartilhamento.

2 - Metodologia

A implementação feita para o primeiro teste que é requisitado na tarefa já havia sido feito na Tarefa 6, com isso foi criada uma implementação usando um vetor compartilhado no arquivo `parallelpi_v`, após isso foram feitos os primeiros testes usando as duas implementações com o `rand()`, após isso foi utilizado o `rand_r()` para criar duas novas versões a `parallelpif.c` que é a versão na qual não é utilizada o vetor compartilhado e `parallelpi_vf.c` essa que seria a versão usando o vetor compartilhado e o `rand_r()`. Na compilação não foi utilizado nenhum tipo de otimização no código e na execução foram usados às 12 threads que a máquina dispunha. Abaixo pode ser visto como foi feita a compilação e execução:

```
gcc -fopenmp parallelpi.c -o parallelpi
export OMP_NUM_THREADS=12; time ./parallelpi

gcc -fopenmp parallelpif.c -o parallelpif
export OMP_NUM_THREADS=12; time ./parallelpif

gcc -fopenmp parallelpi_v.c -o parallelpi_v
export OMP_NUM_THREADS=12; time ./parallelpi_v

gcc -fopenmp parallelpi_vf.c -o parallelpi_vf
export OMP_NUM_THREADS=12; time ./parallelpi_vf
```

2.3 - Ambiente de Execução

Componente	Especificação
Notebook	Lenovo IdeaPad 3 15ALC6
SO	Ubuntu 26.04 LTS
Kernel	Linux 7.0.0-29-generic
CPU	AMD Ryzen 5 5500U — 6C/12T
Arquitetura	x86_64
RAM	9,05 GiB
Desktop	GNOME 50.1
Shell	Zsh 5.9
Terminal	Kitty 0.45.0
Compilador	GCC

3 - Código Fonte

3.1 - parallelpi.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

int main(){
    long long N = 10000000;
    long long pontos_dentro = 0;
    long long ultimo = -1;
    double x = 0.0,y = 0.0;

    srand((unsigned int)time(NULL));

    double inicio = omp_get_wtime();
    #pragma omp parallel default(none) shared(N, pontos_dentro, ultimo) firstprivate(x,y)
    {
        long long local_dento_c = 0;

        #pragma omp for lastprivate(ultimo)
        for(long long i = 0; i < N; i++){
            //Gear as coordenadas de x e y entre 0 e 1
            x = (double)rand()/RAND_MAX;
            y = (double)rand()/RAND_MAX;

            if ((x * x) + (y * y) <= 1.0)
            {
                local_dento_c++;
            }

            ultimo = i;
        }
    }
}
```

```

    }

    #pragma omp critical

    pontos_dentro += local_dento_c;
}

double fim = omp_get_wtime();

double pi_estimado = 4.0 * ((double)pontos_dentro / N);

printf("Pontos sorteados: %lld\n", N);
printf("Pontos dentro do círculo: %lld\n", pontos_dentro);
printf("Valor estimado de Pi: %f\n", pi_estimado);
printf("Tempo Medido: %f (s) \n", fim - inicio);

return 0;
}

```

3.2 - parallelpi_v.c

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

int main(){
    long long N = 10000000;
    long long pontos_dentro = 0;
    long long ultimo = -1;
    double x = 0.0,y = 0.0;

```

```

srand((unsigned int)time(NULL));

int num_threads = omp_get_max_threads();

long long *acertos = calloc(num_threads, sizeof(long long));

double inicio = omp_get_wtime();

#pragma omp parallel default(none) shared(N, pontos_dentro, ultimo, acertos,
num_threads) firstprivate(x,y)
{
    long long local_dento_c = 0;

    int tid = omp_get_thread_num();

    #pragma omp for lastprivate(ultimo)
    for(long long i = 0; i < N; i++){
        //Gear as coordenadas de x e y entre 0 e 1
        x = (double)rand()/RAND_MAX;
        y = (double)rand()/RAND_MAX;

        if ((x * x) + (y * y) <= 1.0)
        {
            local_dento_c++;
        }

        ultimo = i;
    }

    acertos[tid] = local_dento_c;
}

double fim = omp_get_wtime();

for(int i = 0; i < num_threads; i++){

```

```

    pontos_dentro += acertos[i];
}

double pi_estimado = 4.0 * ((double)pontos_dentro / N);

printf("Pontos sorteados: %lld\n", N);
printf("Pontos dentro do círculo: %lld\n", pontos_dentro);
printf("Valor estimado de Pi: %f\n", pi_estimado);
printf("Tempo Medido: %f (s) \n", fim - inicio);

free(acertos);
return 0;
}

```

3.3 - parallelpif.c

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

int main(){
    long long N = 10000000;
    long long pontos_dentro = 0;
    long long ultimo = -1;
    double x = 0.0,y = 0.0;

```

```
double inicio = omp_get_wtime();

#pragma omp parallel default(none) shared(N, pontos_dentro, ultimo) firstprivate(x,y)
{
    long long local_dento_c = 0;
    int tid = omp_get_thread_num();
    unsigned int seed = (unsigned int)time(NULL) ^ tid;

    #pragma omp for lastprivate(ultimo)
    for(long long i = 0; i < N; i++){
        //Gear as coordenadas de x e y entre 0 e 1
        x = (double)rand_r(&seed)/RAND_MAX;
        y = (double)rand_r(&seed)/RAND_MAX;

        if ((x * x) + (y * y) <= 1.0)
        {
            local_dento_c++;
        }
        ultimo = i;
    }
    #pragma omp critical
    pontos_dentro += local_dento_c;
}

double fim = omp_get_wtime();

double pi_estimado = 4.0 * ((double)pontos_dentro / N);

printf("Pontos sorteados: %lld\n", N);
printf("Pontos dentro do círculo: %lld\n", pontos_dentro);
printf("Valor estimado de Pi: %f\n", pi_estimado);
```

```

printf("Tempo Medido: %f (s) \n", fim - inicio);

return 0;
}

```

3.4 - parallempi_vf.c

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

int main(){
    long long N = 10000000;
    long long pontos_dentro = 0;
    long long ultimo = -1;
    double x = 0.0,y = 0.0;

    int num_threads = omp_get_max_threads();
    long long *acertos = calloc(num_threads, sizeof(long long));

    double inicio = omp_get_wtime();
    #pragma omp parallel default(none) shared(N, pontos_dentro, ultimo, acertos,
num_threads) firstprivate(x,y)
    {
        long long local_dento_c = 0;
        int tid = omp_get_thread_num();
        unsigned int seed = (unsigned int)time(NULL) ^ tid;
    }
}

```

```
#pragma omp for lastprivate(ultimo)

for(long long i = 0; i < N; i++){

    //Gear as coordenadas de x e y entre 0 e 1

    x = (double)rand_r(&seed)/RAND_MAX;
    y = (double)rand_r(&seed)/RAND_MAX;

    if ((x * x) + (y * y) <= 1.0)
    {
        local_dento_c++;
    }
    ultimo = i;
}

acertos[tid] = local_dento_c;
}

double fim = omp_get_wtime();

for(int i = 0; i < num_threads; i++){
    pontos_dentro += acertos[i];
}

double pi_estimado = 4.0 * ((double)pontos_dentro / N);

printf("Pontos sorteados: %lld\n", N);
printf("Pontos dentro do círculo: %lld\n", pontos_dentro);
printf("Valor estimado de Pi: %f\n", pi_estimado);
printf("Tempo Medido: %f (s) \n", fim - inicio);

free(acertos);
```

```
return 0;
}
```

4 - Resultado

Após realizada a organização dos 4 arquivos separados o primeiro arquivo executado foi o mesmo usado para a tarefa 6 parallelpi, e pode ser observada a tarefa abaixo na figura 1:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_8 _ □ ×
> export OMP_NUM_THREADS=12; time ./parallelpi
Pontos sorteados: 10000000
Pontos dentro do círculo: 7852252
Valor estimado de Pi: 3.140901
Tempo Medido: 2.171972 (s)
./parallelpi 2,68s user 16,63s system 880% cpu 2,192 total
~ / Doc / g / Programacao_Paralela / Tarefa_8 on main
```

Figura 1: Saída parallelpi.c

O arquivo que utiliza o vetor compartilhado foi o segundo a ser executado e pode ser observada a sua saída na figura 2:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_8 _ □ ×
> export OMP_NUM_THREADS=12; time ./parallelpi_v
Pontos sorteados: 10000000
Pontos dentro do círculo: 7854151
Valor estimado de Pi: 3.141660
Tempo Medido: 2.180675 (s)
./parallelpi_v 2,60s user 16,66s system 881% cpu 2,185 total
~ / Doc / g / Programacao_Paralela / Tarefa_8 on main
```

Figura 2: Saída parallelpi_v.c

O passo seguinte foi testar o parallelpil.c usando o rand_r(), abaixo a saída do mesmo:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_8 _ □ X
> export OMP_NUM_THREADS=12; time ./parallelpif
Pontos sorteados: 10000000
Pontos dentro do círculo: 7853550
Valor estimado de Pi: 3.141420
Tempo Medido: 0.025327 (s)
./parallelpif 0,28s user 0,00s system 948% cpu 0,030 total
```

Figura 3: Saída parallelpif.c

Por fim foi feita a execução da ultima versão que seria a utilizando o vetor compartilhado com o rand_r() que pode ser observado abaixo:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_8 _ □ X
> export OMP_NUM_THREADS=12; time ./parallelpi_vf
Pontos sorteados: 10000000
Pontos dentro do círculo: 7855775
Valor estimado de Pi: 3.142310
Tempo Medido: 0.031761 (s)
./parallelpi_vf 0,32s user 0,00s system 908% cpu 0,036 total
```

Figura 4: Saída parallelpi_vf.c

Cada teste foi executado individualmente evitando ao máximo a execução de uma após a outra de forma a gerar algum acréscimo de desempenho ou de reduz, por fim foi gerada a tabela abaixo com os dados de saída de cada execução para que possa ser comparada às execuções com problema de race condition e como a sua resolução, resolve em sua o desempenho do programa.

Tabela 1: Resultados dos Testes

Código de executado	P. Dentro	Valor de pi	T. Medido	SpeedUp
parallelpif.c	7852252	3,140901	2,171972	0,07163306
parallelpi_v.c	7854151	3,141660	2,180675	0,071347175
parallelpif.c	7853550	3,141420	0,025327	6,14304892
parallelpi_vf.c	7855775	3,142310	0,031761	4,898617802
Sequence.pi	-----	-----	0,155585	-----

5 - Conclusão

A realização desta tarefa permitiu analisar, na prática, os efeitos da concorrência, da sincronização e da geração de números aleatórios em programas paralelos, a comparação entre as quatro implementações mostrou que a escolha da função que será utilizada pode influenciar diretamente no desempenho do sistema, e também que escolher as cláusulas de maneira adequada ajudar a distribuir de maneira igualitária os dados entre as threads.

Nas versões que utilizam `rand()`, os tempos de execução foram próximos de $t=2,18$ s, já com `rand_r()` fica em torno de $t = 0,03$ s. Essa diferença evidencia que o uso de `rand()` em um ambiente multithread pode introduzir um forte custo de sincronização associado ao estado compartilhado utilizado pela função. Já `rand_r()` permite que cada thread mantenha sua própria semente, eliminando completamente essa serialização e permitindo que a geração dos números ocorra de fato em paralelo.

Em relação à coerência de cache e ao falso compartilhamento, o vetor `acertos` é alocado como um bloco contíguo de memória (via `calloc`), e por isso diferentes posições ocupadas por threads distintas podem cair na mesma linha de cache (tipicamente 64 bytes, o suficiente para 8 posições de `long long`). Isso gera, em teoria, falso compartilhamento quando uma thread escreve em sua posição do vetor, a linha de cache correspondente é invalidada nos núcleos das demais threads, mesmo que elas estejam escrevendo em posições diferentes da mesma linha. No entanto, como no código cada thread escreve em `acertos[tid]` apenas uma vez, ao final do laço, esse efeito ocorre no máximo `num_threads` vezes durante toda a execução um custo muito pequeno diante do tempo total do programa. O ganho de desempenho realmente expressivo e consistente nas quatro versões veio da substituição de `rand()` por `rand_r()`: a versão `parallelpif.c` (`critical + rand_r`) atingiu o maior speedup registrado, 6,14, e a versão `parallempi_vf.c` (`vetor + rand_r`) atingiu 4,90 ambas ordens de grandeza acima das versões com `rand()` (speedups de aproximadamente 0,07, ou seja, mais lentas que a versão sequencial). Isso confirma que o principal gargalo de desempenho identificado nesta tarefa foi a serialização imposta pelo `rand()`, e não a forma de agregação dos resultados por thread.

Por fim, a tarefa reforçou a importância de compreender os mecanismos de memória e sincronização presentes em arquiteturas multicore. Pequenas decisões de implementação, como a escolha entre `rand()` e `rand_r()` ou entre uma variável global protegida por `critical` e um armazenamento separado por thread, podem produzir diferenças significativas no desempenho de uma aplicação paralela.